UNIVERSIDAD DE MENDOZA – SEDE SAN RAFAEL

Facultad de Ingeniería | Cátedra: Programación I

Trabajo Práctico N° 10

Despliegue Integral de la Aplicación en la Nube

Puesta en producción de Backend REST (Django), Base de Datos PostgreSQL, Frontend SPA (React + Vite) y Almacenamiento Multimedia (Cloudinary CDN).

FoodRush

Proyecto:	FoodRush – Plataforma de Delivery Gastronómico y Comercial
Docente:	Nicolás Carballo
Alumno:	Agustín Valdebenito
Carrera:	Ingeniería en Informática
Ciclo Lectivo:	2026
Infraestructura Cloud:	Render (API & PostgreSQL) • Vercel (React SPA) • Cloudinary (CDN)

1. Introducción y Objetivos del Proyecto

En el marco de la cátedra Programación I, el presente Trabajo Práctico N° 10 tiene como propósito la puesta en producción y despliegue real en la nube de la plataforma integral FoodRush. FoodRush es una solución de comercio electrónico y delivery multicomercio que articula una arquitectura desacoplada: por un lado, una API RESTful desarrollada con Django y Django REST Framework conectada a una base de datos relacional PostgreSQL; y por el otro, una interfaz de usuario interactiva construida en React con Vite y estilizada mediante Bootstrap.

A diferencia del entorno de desarrollo local, llevar una aplicación de estas características a internet demanda abordar problemáticas esenciales de ingeniería de software en entornos productivos: comunicación entre dominios cruzados (CORS y CSRF), persistencia en servidores con sistemas de archivos efímeros, compresión de recursos estáticos, seguridad mediante variables de entorno encriptadas y automatización de despliegue continuo (CI/CD) conectado al repositorio de control de versiones.

Objetivos específicos alcanzados:
Los objetivos técnicos centrales propuestos para este trabajo fueron:

- **Publicación de la API Backend en Render:** Configuración de un entorno WSGI de producción con Gunicorn, ejecución automatizada de migraciones y orquestación con Render Web Services.
- **Aprovisionamiento de Base de Datos PostgreSQL:** Creación y conexión de una instancia relacional administrada en la nube con soporte SSL para almacenar usuarios, tiendas, productos, carritos y órdenes.
- **Despliegue del Frontend React en Vercel:** Compilación optimizada mediante Vite en la red Edge global de Vercel, garantizando soporte nativo para rutas SPA mediante reescrituras de servidor.
- **Almacenamiento Multimedia Persistente en Cloudinary:** Integración de un CDN en la nube para resolver la volatilidad de los discos de Render, almacenando y optimizando las fotos del catálogo de comidas y locales.
- **Monitoreo y Documentación Pública:** Implementación de un endpoint dedicado de comprobación de salud (/api/health/) y publicación de la documentación interactiva Swagger/OpenAPI (/api/docs/).

2. Arquitectura de Despliegue y Servicios Cloud Utilizados

Para responder a los requerimientos de disponibilidad y alto rendimiento del proyecto, se adoptó una arquitectura de servicios especializados por capa (Multi-Cloud Especializado):

Capa / Componente	Proveedor Cloud	Tecnología Empleada	Rol Técnico en FoodRush
Frontend Web	Vercel	React 18 + Vite (SPA)	Distribución en CDN global (Edge), tiempo de carga instantáneo y soporte de rutas.
Backend API	Render	Python 3.12 + Django + Gunicorn	Servidor WSGI para procesar la lógica de negocio, endpoints REST y autenticación JWT.
Base de Datos	Render	PostgreSQL 16 Administrado	Persistencia ACID de modelos relacionales, vinculada internamente en la región Oregon.
Archivos Multimedia	Cloudinary	Cloudinary Storage & CDN	Servidor externo de imágenes para evitar pérdida de fotos ante reinicios del contenedor.
Integración Continua	GitHub Actions	Workflows CI + Triggers Git	Pipelines automatizados de linting, testing unitario y despliegue automático ante push a main.

3. Entregables y Enlaces Públicos en Producción

A continuación, se listan los accesos directos verificados del entorno productivo:

- **Frontend en Producción (Vercel):** https://foodrush-delivery.vercel.app/
- **API Backend Base (Render):** https://programacion-i-proyecto.onrender.com/
- **Documentación Swagger / OpenAPI:** https://programacion-i-proyecto.onrender.com/api/docs/
- **Endpoint de Monitoreo (Health Check):** https://programacion-i-proyecto.onrender.com/api/health/
- **Servidor CDN Multimedia (Cloudinary):** https://res.cloudinary.com/ed4lhzzl/image/upload/...
- **Repositorio Monorepo en GitHub:** https://github.com/AgusValdebenito/Programacion-I-Proyecto

4. Evidencias del Funcionamiento en Producción

En esta sección se presentan las capturas de pantalla que acreditan la puesta en marcha exitosa, la conectividad integral y la verificación de las diferentes capas del ecosistema FoodRush en internet.

4.1. Aplicación Frontend Desplegada y Operativa

La interfaz web construida en React consume los endpoints del backend en Render en tiempo real. En la portada se aprecia la resolución correcta de comercios, categorías interactivas y recursos de diseño con el favicon personalizado de la marca:

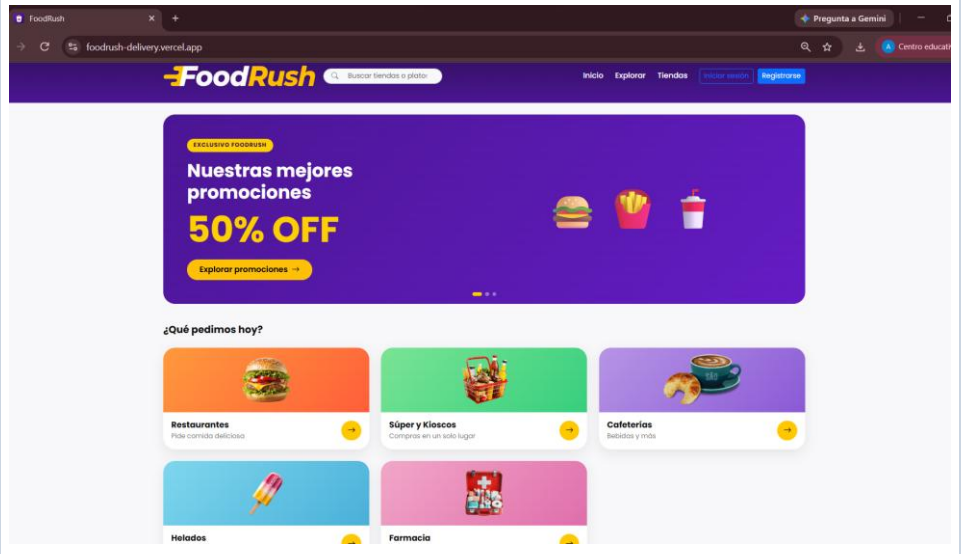


Figura 1: *Vista principal de FoodRush en producción alojada en Vercel con carga dinámica de comercios.*

Al seleccionar un comercio, la aplicación consulta la carta de productos asociada al local, cargando imágenes de platos optimizadas desde Cloudinary y permitiendo agregar artículos al carrito:

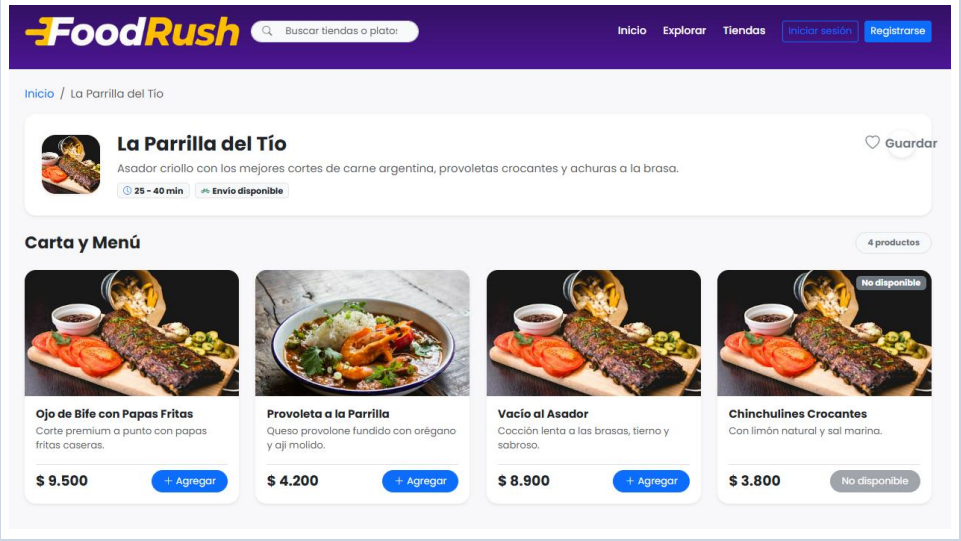


Figura 2: *Carta de productos de comercio adherido con detalle de ítems y control de stock.*

4.2. Documentación Interactiva Swagger / OpenAPI

La API expone públicamente su especificación OpenAPI 3.0 mediante drf-spectacular en la ruta /api/docs/, permitiendo inspeccionar y ejecutar pruebas de cada uno de los ViewSets del sistema (tiendas, productos, órdenes, autenticación y notificaciones):

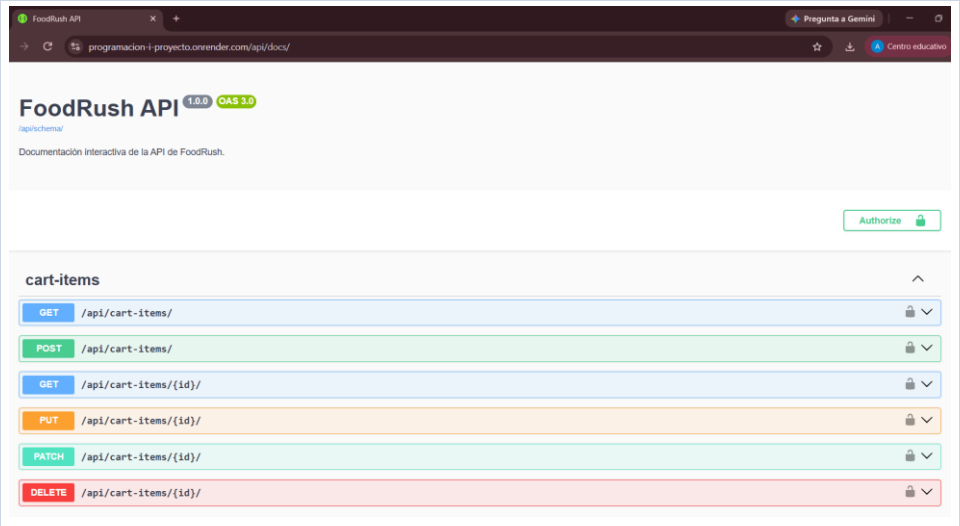


Figura 3: *Interfaz de Swagger UI expuesta en Render para exploración de la API REST.*

4.3. Endpoint de Monitoreo y Verificación de Salud (/api/health/)

Siguiendo las mejores prácticas de arquitectura en la nube, se creó el endpoint público GET /api/health/. Este servicio responde de manera liviana con HTTP 200 OK y el cuerpo JSON {"status": "ok", "service": "api"}, permitiendo certificar que la aplicación está en línea:

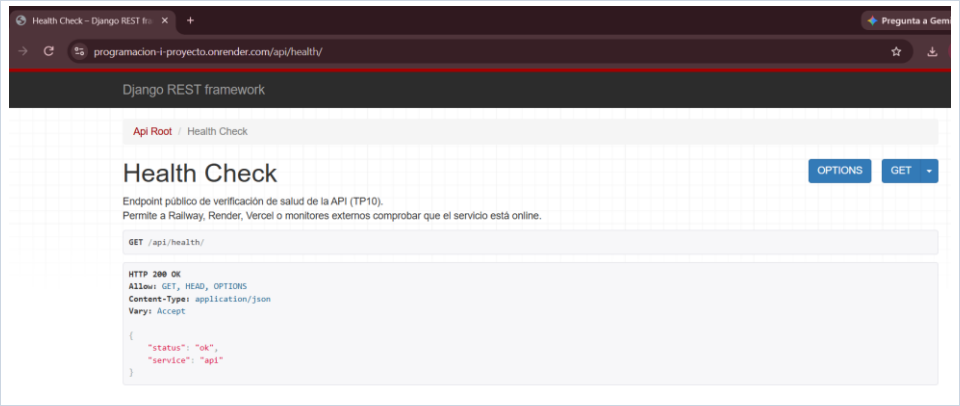


Figura 4: *Respuesta satisfactoria del endpoint de monitoreo /api/health/ en producción.*

4.4. Control de Acceso y Autenticación JWT

La API implementa seguridad robusta basada en JSON Web Tokens (SimpleJWT). A continuación se demuestra el comportamiento del sistema ante peticiones anónimas y peticiones autenticadas:

a) Acceso Restringido (Error 401 Unauthorized): Al intentar acceder a recursos sensibles como la consulta de pedidos (/api/orders/) sin proporcionar la cabecera 'Authorization: Bearer <token>', la API deniega la operación de forma inmediata con código 401:

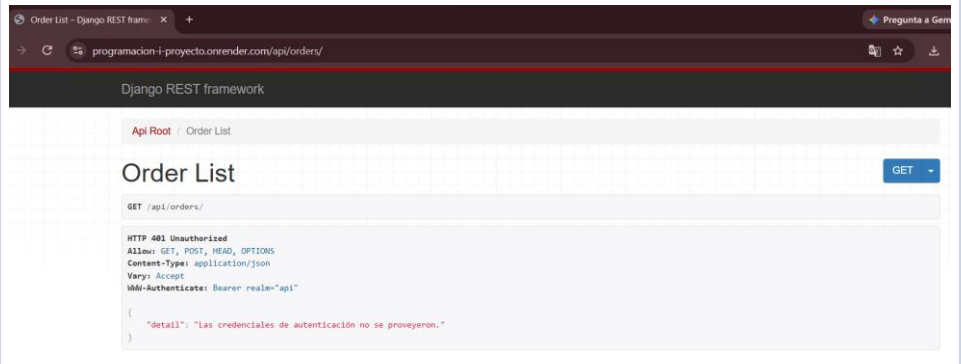


Figura 5: *Respuesta HTTP 401 Unauthorized ante intento de acceso no autenticado.*

b) Autenticación Válida y Emisión de Tokens (HTTP 200 OK): Al enviar credenciales válidas (email y contraseña) mediante POST a /api/token/, la API valida la identidad contra PostgreSQL y devuelve los tokens criptográficos access y refresh junto a la información de rol del usuario:

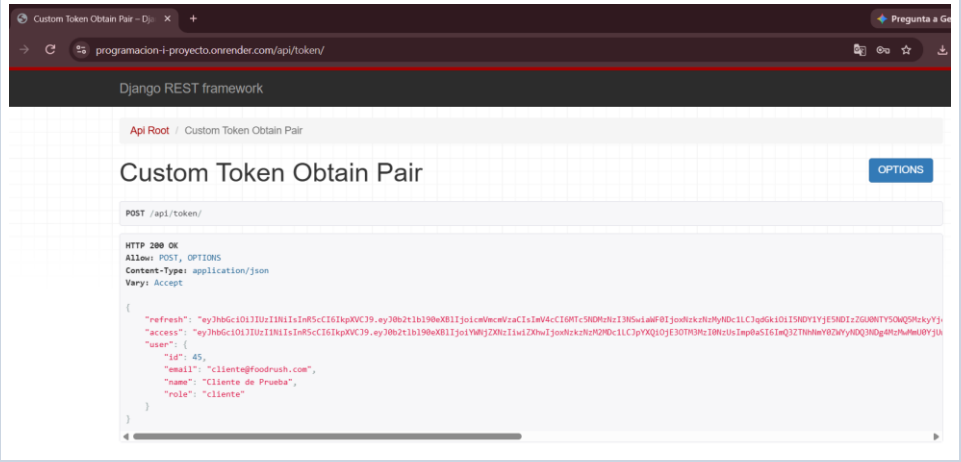


Figura 6: *Inicio de sesión exitoso con emisión de tokens JWT de acceso y refresco.*

4.5. Paneles de Control de los Proveedores Cloud

Para validar la correcta configuración de la infraestructura, se adjuntan las vistas de los tableros de control en cada plataforma:

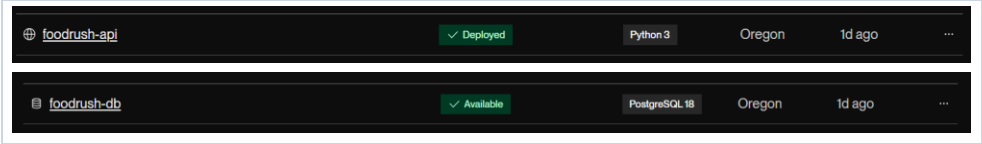


Figura 7: *Dashboard de Render mostrando el backend WSGI y la base de datos PostgreSQL en ejecución activa ('Live').*

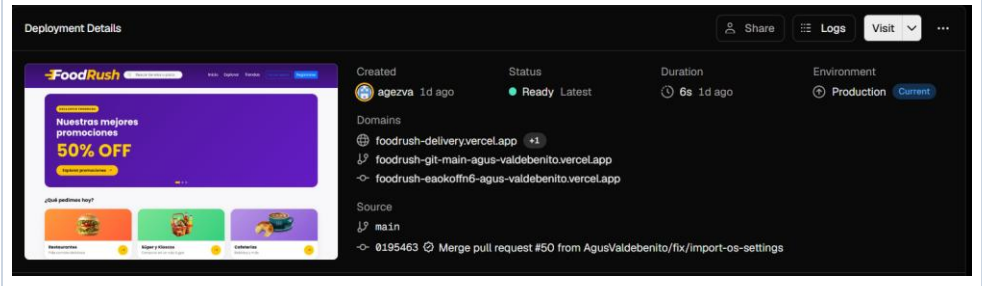


Figura 8: *Dashboard de Vercel con el despliegue automático del frontend en estado 'Ready'.*

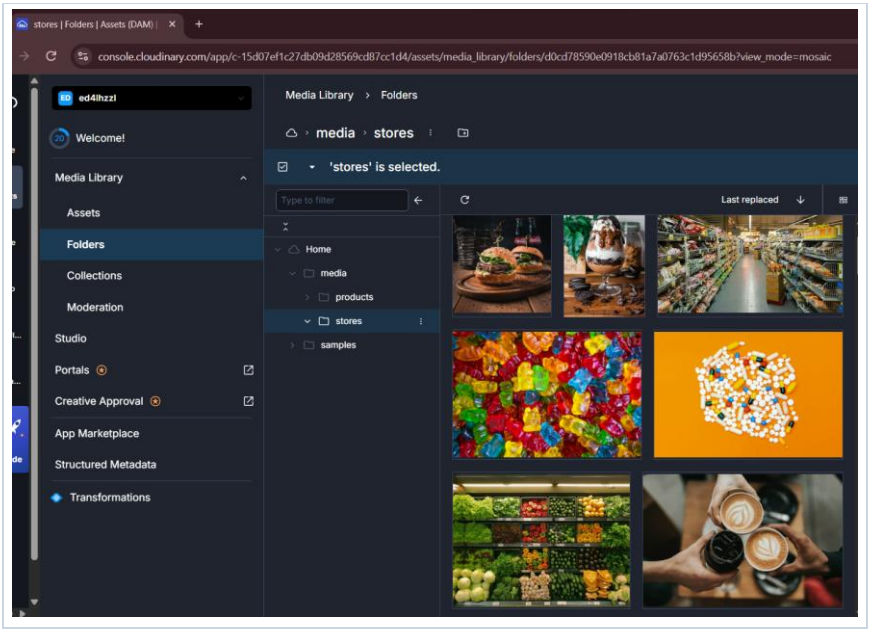


Figura 9: *Biblioteca multimedia de Cloudinary alojando de forma persistente los recursos visuales del proyecto.*

5. Proceso Técnico de Despliegue: "Cómo se subió"

A continuación se detalla el procedimiento técnico cronológico y las configuraciones de código aplicadas para lograr el despliegue exitoso de cada componente:

5.1. Despliegue del Backend Django y PostgreSQL en Render

1. Preparación de dependencias en requirements.txt: Se añadieron paquetes indispensables para producción: gunicorn (servidor WSGI HTTP), whitenoise (compresión y servicio de estáticos), dj-database-url (interpretación dinámica de cadenas de conexión PostgreSQL), y django-cloudinary-storage junto a cloudinary (driver de almacenamiento en la nube).

2. Aprovisionamiento de PostgreSQL en Render: Desde el panel de Render se creó la base de datos relacional administrada 'foodrush-db' en la región Oregon (US West). Render suministró la variable de entorno DATABASE_URL protegida con SSL que vincula al servidor backend sin exponer contraseñas en el código fuente.

3. Creación y configuración del Web Service: Se conectó el repositorio de GitHub con runtime Python 3. Se establecieron los siguientes parámetros clave de orquestación:

• **Comando de Construcción (Build Command):** `pip install -r requirements.txt && python manage.py collectstatic --noinput && python manage.py upload_media_to_cloudinary`

• **Comando de Inicio (Start Command):** `python manage.py migrate --noinput && python manage.py populate_extended_catalog && python manage.py upload_media_to_cloudinary && gunicorn FoodRush.wsgi:application`

4. Variables de Entorno inyectadas en Render:
• DJANGO_SECRET_KEY: Clave criptográfica única y secreta de producción.
• DJANGO_DEBUG: Establecida en 'False' para apagar el modo de depuración en internet.
• DJANGO_ALLOWED_HOSTS: '127.0.0.1,localhost,.onrender.com'.
• DATABASE_URL: Inyectada automáticamente desde la base de datos PostgreSQL.
• CLOUDINARY_URL: Cadena de conexión provista por Cloudinary (cloudinary://API_KEY:API_SECRET@ed4lhzzl).
• PYTHON_VERSION: '3.12.8'.

5. Adecuaciones en FoodRush/settings.py: Se insertó WhiteNoiseMiddleware en la posición inmediata inferior a SecurityMiddleware, se definió CompressedManifestStaticFilesStorage, y se flexibilizó la política de orígenes seguros mediante expresiones regulares para autorizar cualquier subdominio emitido por Vercel:
CORS_ALLOWED_ORIGIN_REGEXES = [r"^https:\/\/.*\.vercel\.app$", r"^https:\/\/.*\.onrender\.com$"]
CSRF_TRUSTED_ORIGINS = ['https://*.onrender.com', 'https://*.vercel.app']

5.2. Despliegue del Frontend React en Vercel

1. Conexión del Repositorio y Subcarpeta (Monorepo): Dado que el repositorio contiene tanto el backend como el frontend, en la configuración de Vercel se definió el 'Root Directory' apuntando exclusivamente a la subcarpeta 'frontend/'.

2. Detección de Build y Entorno Vite: Vercel reconoció automáticamente el preset Vite, asignando el comando 'npm run build' y definiendo la carpeta de salida 'dist/'.

3. Variable de Entorno de Conexión: Se configuró la variable VITE_API_URL = 'https://programacion-i-proyecto.onrender.com/api'. Durante la compilación, Vite inyecta esta dirección en los servicios de Axios/Fetch.

4. Enrutamiento SPA con vercel.json: Para solucionar el problema clásico de las aplicaciones de una sola página (donde al recargar rutas como /tiendas o /perfil el servidor devuelve 404), se configuró una regla de reescritura en vercel.json redirigiendo todas las peticiones entrantes a /index.html.

5.3. Gestión y Persistencia de Medios con Cloudinary

Uno de los principales desafíos técnicos detectados fue que en proveedores como Render o Heroku el sistema de archivos del contenedor es efímero (stateless). Cualquier imagen subida a disco local se destruye en cada reinicio o pausa por inactividad.

Para dar solución integral a este requerimiento se implementó:
1. Integración de django-cloudinary-storage en los modelos de Store y Product para que las URLs públicas apunten al CDN global de Cloudinary.
2. Comando de migración upload_media_to_cloudinary: Script que recorre el catálogo inicial local, verifica si la imagen ya existe en Cloudinary mediante peticiones HEAD ligeras para no desperdiciar cuota, y sube los archivos pendientes asignando su public_id exacto.
3. Resiliencia en Frontend (Fallback Automático): En imageUrl.js se programó un manejador global de errores en fase de captura. Si por problemas de red o latencia un recurso de Cloudinary demora o arroja error, el cliente redirige inmediatamente la petición al backend, evitando pantallas con imágenes rotas.

6. Conclusión y Aprendizajes Técnicos

La realización de este trabajo práctico permitió consolidar conceptos fundamentales de DevOps y despliegue continuo aplicados a un proyecto de software moderno. Lograr que una API Django, una base de datos PostgreSQL, un frontend React y un CDN multimedia interactúen de forma segura y automatizada en internet constituyó una experiencia formativa sumamente enriquecedora.

Entre los desafíos técnicos más destacados resueltos con éxito se encuentran la resolución de políticas de seguridad entre orígenes cruzados (CORS/CSRF), el empaquetado de recursos estáticos con WhiteNoise sin impactar el rendimiento, la persistencia en la nube frente a contenedores efímeros, y el desacoplamiento efectivo entre cliente y servidor. Como resultado final, FoodRush se encuentra completamente desplegado, accesible para cualquier usuario en internet y respaldado por una infraestructura sólida y escalable.